ConstructaIQ

Backend API Documentation

Multi-Agent Construction Intelligence Platform

Version 1.0.0 | June 12, 2026

- FastAPI + Azure AI Foundry multi-agent pipeline
- Azure SQL persistence (12 tables)
- Async project analyze jobs with status polling
- Document Intelligence + Blob Storage integration

Table of Contents

- 1. Executive Summary
- 2. System Architecture
- 3. Technology Stack
- 4. Project Structure
- 5. Configuration and Environment Variables
- 6. Database Schema
- 7. Agent Pipeline
- 8. Persistence Layer
- 9. API Reference
- 10. Async Analyze Jobs
- 11. Azure Service Integrations
- 12. Response Contract (Frontend)
- 13. Testing and Scripts
- 14. Postman Collection
- 15. Known Limitations and Roadmap

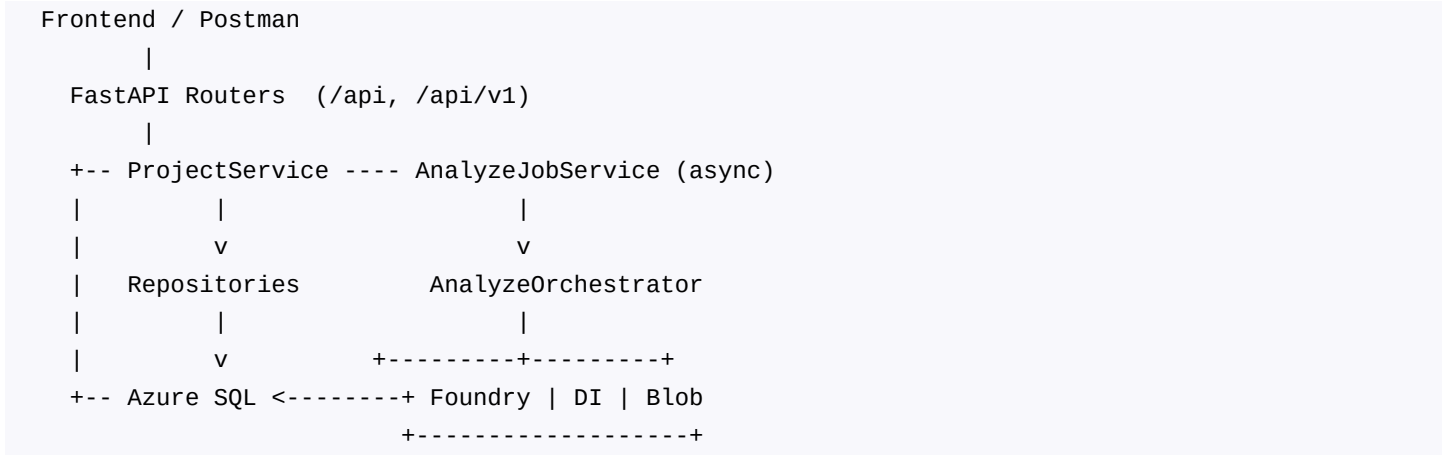
1. Executive Summary

ConstructaIQ Backend is a FastAPI service that orchestrates a multi-agent AI pipeline for construction project onboarding and intelligence. It integrates with Azure AI Foundry to run specialized agents (contract extraction, blueprint analysis, permits, planning, supplier matching, crew allocation, knowledge insights, and project health diagnosis). Results can be persisted to Azure SQL Database and exposed through REST APIs designed for frontend consumption.

The primary integration path is the project lifecycle flow: create or upload a project, enqueue an async analyze job (HTTP 202), poll for completion, then read structured intelligence data (summary, health, suppliers, crew, agent audit trail). Stateless shortcuts remain available for direct pipeline invocation without a project record.

2. System Architecture

The backend follows a layered architecture: API routers delegate to service classes, which coordinate the orchestrator, persistence, and Azure integrations. The analyze orchestrator runs agents sequentially, passing JSON context from each step to the next.



Core Components

- app/main.py - FastAPI app, CORS, lifespan (DB init/dispose), router mounting
- app/api/projects.py - Project lifecycle and read endpoints
- app/api/analyze.py - Stateless Mode 1 (text) and Mode 2 (documents)
- app/api/catalogs.py - Supplier and crew master catalog CRUD
- app/orchestrator/analyze_orchestrator.py - Multi-agent pipeline
- app/services/agent_persistence_service.py - DB persistence after pipeline
- app/services/analyze_job_service.py - In-memory async job queue
- app/services/response_mapper.py - DB to frontend DTO mapping
- app/db/ - SQLAlchemy models, async repositories, Alembic migrations

3. Technology Stack

Component	Technology
Web framework	FastAPI 0.111+
Server	Uvicorn

Component	Technology
ORM	SQLAlchemy 2.0 (async)
Database driver	aioodbc + pyodbc (Azure SQL)
Migrations	Alembic
Validation	Pydantic v2
AI agents	Azure AI Foundry REST API
Document OCR	Azure Document Intelligence
File storage	Azure Blob Storage (optional)
Testing	pytest, pytest-asyncio, FastAPI TestClient

4. Project Structure

```
backend/
  app/
    main.py                 Application entry point
    config/settings.py      Environment configuration
    api/                    REST route handlers
    orchestrator/           Agent pipeline
    services/               Business logic
    db/models/              SQLAlchemy ORM models
    db/repositories/        Data access layer
    schemas/                Pydantic request/response DTOs
  alembic/                  Database migrations
  scripts/                  Seed, E2E, and utility scripts
  tests/                    Unit tests
  postman/                  Postman collection and environment
  docs/                     Generated documentation (this PDF)
```

5. Configuration and Environment Variables

All settings are loaded from backend/.env via pydantic-settings.

Variable	Required	Description
AZURE_AIFOUNDRY_ENDPOINT	Yes	Foundry REST endpoint URL
AZURE_AIFOUNDRY_KEY	Yes	Foundry API key
AZURE_DOCUMENT_INTELLIGENCE_ENDPOINT	Mode 2	Document Intelligence endpoint
AZURE_DOCUMENT_INTELLIGENCE_KEY	Mode 2	Document Intelligence key
AZURE_STORAGE_CONNECTION_STRING	Optional	Blob storage connection
AZURE_STORAGE_CONTAINER_NAME	Optional	Blob container name
DATABASE_URL	Optional	Azure SQL async connection string
PERMIT_AGENT_VERSION	Optional	Default: 3
SKIP_RISK_RECOVERY_DEFAULT	Optional	Default: true

Variable	Required	Description
PORT	Optional	Default: 8000

Setup Commands

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
python -m alembic upgrade head
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

6. Database Schema

12 tables migrated via Alembic (001_baseline, 002_add_core_tables). The app runs without a database when DATABASE_URL is unset. Project APIs require DB.

Table	Model	Purpose
projects	Project	Root construction project
documents	Document	Uploaded PDFs and blob paths
agent_executions	AgentExecution	Per-agent run audit trail
permits	Permit	Permit assessments
schedules	Schedule	Phase breakdown and duration
project_suppliers	ProjectSupplier	Procurement plan rows
crew_plans	CrewPlan	Crew allocations by phase
inspections	Inspection	Inspection checkpoints
budgets	Budget	Cost breakdown
supplier_master	SupplierMaster	Supplier directory
supplier_materials	SupplierMaterial	Supplier catalog items
crew_master	CrewMaster	Workforce directory

Key Project Columns (projects table)

- project_id (PK), project_name, project_type, location, client_name, status
- start_date, target_completion_date, contract_value, duration_months
- scope, milestones, square_footage, floor_count, complexity_level, priority_score

7. Agent Pipeline

The orchestrator (analyze_orchestrator.py) runs agents sequentially. Each agent receives JSON context from prior steps. All prompts append JSON_OUTPUT_SUFFIX to request valid JSON-only responses. Each _call() records started_at, completed_at, and output in an execution_log for agent_executions persistence.

Active Pipeline (skip_risk_recovery=true, default)

```
ContractAgent -> BlueprintAgent -> PermitAgent (v3)
  -> PlanningAgent -> SupplierAgent -> CrewAgent
  -> ConstructionKnowledgeAgent -> DoctorAgent
```

Skipped Agents (not deployed in Foundry)

- RiskAgent - returns skipped payload when skip_risk_recovery=true
- RecoveryAgent - returns skipped payload when skip_risk_recovery=true

Pipeline Modes

- Mode 1 (Text): project_name + description, no documents required

- Mode 2 (Documents): PDF upload, Document Intelligence extraction, then agents

Catalog Injection

SupplierAgent and CrewAgent receive live catalogs from supplier_master, supplier_materials, and crew_master tables via master_catalog_service.py. Seed with: python scripts/seed_supplier_master.py and seed_crew_master.py

Pipeline Output Keys

Key	Source Agent
projectSummary	ContractAgent
blueprintSummary	BlueprintAgent
permitAssessment	PermitAgent
projectPlan	PlanningAgent
supplierAnalysis	SupplierAgent
crewAnalysis	CrewAgent
knowledgeInsights	ConstructionKnowledgeAgent
riskAnalysis	RiskAgent (or skipped)
recoveryPlan	RecoveryAgent (or skipped)
projectHealth	DoctorAgent

8. Persistence Layer

persist_analysis_outputs() in agent_persistence_service.py runs after the full pipeline when project_id and a DB session are provided. Uses replace-on-rerun for derived tables and append-only for agent_executions.

Table	Strategy	Pipeline Source
projects	Update in place	projectSummary fields
permits	Delete + insert	permitAssessment.required_permits
schedules	Delete + insert	projectPlan phases, duration
budgets	Delete + insert	projectSummary.budget + cost sums
inspections	Delete + insert	projectPlan.inspection_stages
project_suppliers	Delete + insert	supplierAnalysis.procurement_plan
crew_plans	Delete + insert	crewAnalysis.crew_allocations
agent_executions	Append only	execution_log per _call()

Date Parsing

- ISO format: 2026-08-01
- Relative: Month N (offset from project.start_date)
- Unparseable dates stored as NULL with warning log

9. API Reference

Routers mounted at /api and /api/v1. Project endpoints return ApiResponse wrapper: { "data": T, "status": "success", "timestamp": "..." }. Stateless /analyze endpoints return raw camelCase JSON.

9.1 Infrastructure

Method	Path	Description
GET	/healthz	Liveness probe
GET	/healthz/db	Database connectivity (503 if down)

9.2 Project Lifecycle

Method	Path	Status	Description
POST	/api/v1/projects	201	Create project (JSON body)
POST	/api/v1/projects/upload	201	Multipart PDF upload
POST	/api/v1/projects/{id}/analyze	202	Enqueue async analyze job
GET	/api/v1/projects/{id}/analyze/status	200	Poll job status
GET	/api/v1/projects/{id}/upload-session	200	UploadSessionResponse shape

9.3 Project Read Endpoints

Method	Path	Description
GET	/api/v1/projects/{id}	Project metadata
GET	/api/v1/projects/{id}/summary	Intelligence view (permits, phases, crew)
GET	/api/v1/projects/{id}/suppliers	Procurement rows
GET	/api/v1/projects/{id}/crew	Crew plan rows
GET	/api/v1/projects/{id}/health	Latest DoctorAgent output
GET	/api/v1/projects/{id}/agents	Agent execution audit trail
GET	/api/v1/projects/{id}/risks	503 stub until RiskAgent deployed
GET	/api/v1/projects/{id}/recovery	503 stub until RecoveryAgent deployed
POST	/api/v1/projects/call-agent	Direct single-agent invocation

9.4 Analyze (Stateless)

Method	Path	Body	Notes
POST	/api/v1/analyze/text	form-data	Sync ~2 min; skip_risk_recovery supported
POST	/api/v1/analyze/documents	multipart PDF	Requires Document Intelligence

9.5 Catalog CRUD

Method	Path	Description
GET/POST	/api/v1/suppliers	List / create suppliers
GET/POST	/api/v1/suppliers/{id}/materials	List / create materials
GET/POST	/api/v1/crew	List / create crew members

9.6 Example: Create Project

```
POST /api/v1/projects
Content-Type: application/json

{
  "project_name": "Tower A",
  "location": "Chicago, IL",
  "scope": "42-floor mixed-use tower...",
  "duration_months": 28,
  "floor_count": 42
}
```

9.7 Example: Start Analyze

```
POST /api/v1/projects/1/analyze
Content-Type: application/json

{
  "description": "Project scope text...",
  "agent_version": "1",
  "skip_risk_recovery": true
}

Response (202): { "data": { "job_id": "uuid", "status": "queued" } }
```

10. Async Analyze Jobs

POST /projects/{id}/analyze returns HTTP 202 immediately. analyze_job_service.py runs the pipeline in a background asyncio task. Jobs are stored in-memory (lost on server restart). Poll GET /projects/{id}/analyze/status until status is complete or error.

Status	Meaning
queued	Job created, not yet started
running	Pipeline in progress; check progress_step and overall_pct
complete	Result available in result field
error	Pipeline failed; error field contains message

Progress steps: ContractAgent, BlueprintAgent, PermitAgent, PlanningAgent, SupplierAgent, CrewAgent, ConstructionKnowledgeAgent, DoctorAgent.

11. Azure Service Integrations

Azure AI Foundry

foundry_service.py calls agents via REST. Each agent has a name and version. PermitAgent uses version from PERMIT_AGENT_VERSION setting (default 3).

Document Intelligence

document_intelligence.py extracts text from PDF bytes or blob SAS URLs for Mode 2 pipeline.

Blob Storage

blob_storage_service.py uploads PDFs to a private container, generates short-lived SAS URLs for DI, and supports download for deferred project analyze.

12. Response Contract (Frontend)

response_mapper.py maps DB entities to frontend TypeScript shapes:

- ProjectIntelligenceData - summary endpoint (permits, phases, crewRequirements)
- Project health - DoctorAgent fields from latest agent_execution
- UploadSessionResponse - job status with agentSteps and overallPct
- RiskIntelligenceData - stub until RiskAgent deployed

13. Testing and Scripts

Script / Command	Purpose
pytest tests/	Unit tests (persistence, response mapper)
scripts/e2e_project_flow_test.py	HTTP project CRUD + read APIs
scripts/e2e_backend_test.py	Live Foundry pipeline (text, ~2 min)
scripts/e2e_mode2_documents.py	Mode 2 PDF pipeline
scripts/test_supplier_crew_pipeline.py	Catalog + persistence tests
scripts/seed_supplier_master.py	Seed 4 suppliers, 8 materials
scripts/seed_crew_master.py	Seed 12 crew members
scripts/check_db.py	Verify Azure SQL connectivity
scripts/check_blob.py	Verify blob storage connectivity
verify_backend.py	Delegates to e2e_project_flow_test.py

14. Postman Collection

Import from backend/postman/:

- ConstructaIQ.postman_collection.json - 24 requests in 6 folders
- ConstructaIQ-Local.postman_environment.json - baseUrl, projectId, jobId

Select environment ConstructaIQ - Local in Postman. Variables projectId and jobId auto-capture on Create Project and Start Analyze. Recommended flow: Create Project -> Start Analyze -> Poll Status -> Get Summary.

15. Known Limitations and Roadmap

Item	Status
Frontend integration	UI still uses mock data
RiskAgent / RecoveryAgent	Not deployed in Foundry; 503 stubs
Authentication / RBAC	Not implemented
Job persistence	In-memory only; lost on restart
Extra requirement agents	InspectionAgent, BudgetAgent, CommunicationAgent not built

Recommended Next Steps

- Wire frontend to project upload + async analyze + summary APIs
- Deploy RiskAgent and RecoveryAgent to Azure AI Foundry
- Add auth middleware for Bearer tokens
- Persist analyze jobs to database for production resilience